

```
=====
What is Database Connectivity?
=====
```

**** Database connectivity means connecting a Python application to a database to perform operations such as:**

- ** Insert data**
- ** Read data**
- ** Update data**
- ** Delete data**

**** These four operations are called CRUD Operations.**

- C â€” Create**
- R â€” Read**
- U â€” Update**
- D â€” Delete**

```
=====
## Types of Databases
=====
```

=> Databases are mainly divided into two types:

- 1) Relational Databases**
- 2) Non-Relational Databases**

**** Relational databases store data in table format.**

Ex : MySQL, Oracle, PostGres SQL, SQL Server, SQLite...

**** Non-relational databases store data in document, key-value, graph or other formats.**

Ex : MongoDB, Redis, Casandara, FireBase...

```
=====
What is SQLite?
=====
```

**** SQLite is a lightweight database.**

Note: SQLite does not require:

- 1) DB Server installation**
- 2) Username and Password**
- 3) Port Number**
- 4) Seperate Configuration**

Note: It stores the complete database inside a single file.

**** Python provides built-in support for SQLite using the "sqlite3 module". ****

-----create db table-----

```
import sqlite3

#Establishing DB Connection
connection = sqlite3.connect("students.db")

# Create cursor object to execute SQL Queries
cursor = connection.cursor()

# Using cursor we can execute SQL queries

cursor.execute("""
    create table if not exists students(
        student_id integer primary key autoincrement,
        student_name text,
        student_email text unique,
        student_course text,
        student_fee real
    )
""")

print("Student Table Created Successfully....")

connection.commit()
connection.close()
```

----- insert record-----

```
import sqlite3

#Establishing DB Connection
connection = sqlite3.connect("students.db")

# Create cursor object to execute SQL Queries
cursor = connection.cursor()

# Read student information
student_name = input("Enter Student Name: ")
student_email = input("Enter Student Email: ")
student_course = input("Enter Student Course: ")
student_fee = float(input("Enter Student Fee: "))

sql = "insert into students(student_name, student_email, student_course, student_fee) values(?, ?, ?, ?)"

cursor.execute(sql, (student_name, student_email, student_course, student_fee))

connection.commit()

print("Student inserted Successfully")
connection.close()
```

----- Retrieve students -----

```
import sqlite3

#Establishing DB Connection
connection = sqlite3.connect("students.db")

# Create cursor object to execute SQL Queries
cursor = connection.cursor()

sql = "select * from students"

cursor.execute(sql)

students = cursor.fetchall()

print(type(students))

for student in students:
    print(student)

----- single record -----
```

```
import sqlite3

#Establishing DB Connection
connection = sqlite3.connect("students.db")

# Create cursor object to execute SQL Queries
cursor = connection.cursor()

# Read student information
student_id = int(input("Enter Student ID: "))

sql = "select * from students where student_id =?"

cursor.execute(sql, (student_id,))

student = cursor.fetchone()
print(student)

-----
```

```
=====
Python with MySQL Database
=====
```

@@@ MySQL DB Server & Workbench Setup : <https://www.youtube.com/watch?v=EsAIXPIsyQg>

Step-1 : Install MySQL Connector using virtual environment

```
$ python -m venv venv
```

```
$ venv\Scripts\activate
```

```
$ pip install mysql-connector-python
```

Note: we can configure required libraries in "requirements.txt" file and we can install them

```
$ pip install -r requirements.txt
```

Step-2 : Create python file with required methods

```
get_connection()
```

```
create_table()
```

```
add_student()
```

```
view_students()
```

```
-----py-mysql-app.py-----
```

```
import mysql.connector
```

```
def get_connection():
    try:
        connection = mysql.connector.connect(
            host="localhost",
            port = "3306",
            user="root",
            passwd="root",
            database="student")
        return connection
    except mysql.connector.Error as error:
        print("database conn failed:", error)
        return None
```

```
def create_table():
    connection = get_connection()
    if connection is None:
        return
    cursor = None
    try:
        cursor = connection.cursor()
        sql = """
        CREATE TABLE IF NOT EXISTS students (
            student_id INT PRIMARY KEY AUTO_INCREMENT,
            student_name VARCHAR(100) NOT NULL,
            student_email VARCHAR(100) UNIQUE NOT NULL,
            student_course VARCHAR(100) NOT NULL,
            student_fee DECIMAL(10, 2) NOT NULL
        )
        """
        cursor.execute(sql)
        connection.commit()
    except Exception as error:
        print("Error while creating table,", error)
```

```
def add_student():
    connection = get_connection()
    cursor = connection.cursor()
```

```

student_name = input("Enter student name: ").strip()
student_email = input("Enter student email: ").strip()
student_course = input("Enter student course: ").strip()
student_fee = float(input("Enter student fee: "))

sql = """
    INSERT INTO students
        (student_name, student_email, student_course, student_fee)
    VALUES (%s, %s, %s, %s) \
    """

values = (
    student_name,
    student_email,
    student_course,
    student_fee
)

cursor.execute(sql, values)
connection.commit()
print("Student Inserted...")
connection.close()

def view_all_students():
    connection = get_connection()
    cursor = connection.cursor()

    sql = "select * from students"

    cursor.execute(sql)

    students = cursor.fetchall()
    for student in students:
        print(student)

# create_table()

add_student()

print("-----")

view_all_students()

-----

```

Assignment

Develop a menu-driven Student Management System using Python and MySQL.

The application should connect to a MySQL database and perform the following CRUD operations:

- 1) Add a new student
- 2) View all student2
- 3) View a student by ID
- 4) Update student details
- 5) Delete a student
- 6) Exit the application

Functional Requirements

- Use mysql-connector-python to connect Python with MySQL.
- Create the database table automatically if it does not exist.
- Use separate functions for each CRUD operation.
- Use parameterized queries with %s placeholders.
- Display a proper message when a student record is not found.
- Prevent duplicate email addresses.
- Handle database connection errors and invalid user input.
- Ask for confirmation before deleting a student.
- Close the cursor and database connection after every operation.